openFerment Development Plan

A plain language guide to what we build and how we judge it

October 22 2026

Version 0.2

Start here

We are building an open engineering platform in which a user directs Postdoc, an AI research coordinator. The work should remain understandable from the original question through sources, estimates, calculations, equipment, actual results, and useful shared knowledge.

The recommended first achievement is one investigation that stays understandable when you return to it. You can recover the question, inspect the inputs and their origins, repeat a supported calculation, and see what remains uncertain. Postdoc should be able to use that record, work within limits, and explain what it did.

The first scientific case is now a bounded investigation of cellulosic ethanol: begin with a documented corn-stover case and batch conversion, then add alternatives only where suitable evidence and models exist. Downstream recovery may use a different operating mode. The platform's wider mission remains intact; this case gives us a concrete way to develop and test it.

That first achievement is smaller than the whole vision. It establishes a foundation on which experiments, equipment histories, collaboration, and independent reproduction can build. The full scope remains visible in the coverage map at the end.

Your role Decide whether the work answers a useful question and whether its behavior and explanation make sense. You do not need to choose a database, read code, or write detailed programming instructions.

The development team's role Investigate the existing software, make and explain technical decisions, build the agreed behavior, check it, repair relevant problems, and show you the result. Technical and scientific claims need evidence beyond an attractive demonstration.

The immediate next step Establish a running baseline and select one existing calculation for the first corn-stover investigation. Inspect BioSTEAM and its reference models as possible dependencies after that baseline. The repository already contains useful tools. We should preserve them and complete the missing connections.

The first prepared assignment Save an investigation with a sourced input and a labeled estimate, connect it to an existing balance or recovery calculation, reopen it, amend it, and export an understandable account. A full new ethanol model is not a prerequisite. Section 6 describes what you will review.

This document develops the product brief, roadmap, decisions, and first assignment. Technical Codex prompts and software implementation remain for a later stage. No product code, deployment, or licence terms have been changed to prepare this plan.

How to use this plan

Read Start here and sections 1 through 3 to see the direction and starting point. Section 5 gives the build sequence. Section 6 is the first concrete assignment. Section 7 explains your repeating workflow. The later sections are reference material for decisions, completion, and coverage.

A milestone is a useful outcome large enough to demonstrate. An assignment is a smaller piece of work needed to reach that outcome. A check is evidence that a specific requirement has been met. These terms describe ordinary work, not a technical system you must learn.

The plan contains recommended defaults so development can proceed without asking you about every routine choice. Defaults are proposals for planning, not already adopted legal terms or claims that the software works.

1 The project brief....3
2 What we appear to have now....4
3 One example we will follow....5
4 The promises we must preserve....7
5 The build sequence....8
6 The first prepared assignment....11
7 Your repeating workflow....13
8 Decisions and recommended defaults....14
9 How we decide that work is complete....17
10 What we prepare later....18
Coverage map....19
Sources and status....21

1 The project brief

What we are building

openFerment should help a person turn an engineering question into work they can inspect, test, and improve. Its first domain is bioprocess engineering. A broader architecture can support other process and manufacturing disciplines as appropriate tools and evidence are added.

The Home experience should be a conversation with Postdoc inside a chosen project. The user can inspect the plan, supporting work, spending, unresolved questions, and decisions. Important results also exist as records the user can open directly. Conversation should not be the only way to find them.

Postdoc coordinates specialist agents and software tools. It should choose work suited to the question, keep useful continuity, offer clearly identified estimates, and explain what the evidence supports. More agents are useful only when they improve the work.

The first flagship question is which supported batch-conversion choices are promising for a characterized lignocellulosic material. Corn stover comes first. Switchgrass is a conditional second case, contingent on suitable reference data and a checked model. This is not a promise to optimize any feedstock or to establish the best commercial use of any biomass resource.

Postdoc also needs a deliberate place for ideas. It may propose an unconventional explanation or process change without identifying a particular source in its training. That proposal remains a hypothesis until evidence supports it. Engineering evaluation, experimentation, and a useful idea are distinct achievements.

The platform includes the machinery and manufacturing knowledge needed to make a process real. A useful public contribution may include editable designs, fabrication methods, controls, commissioning, maintenance, observations, economics, and failures. It should help another competent team understand and reproduce the work.

What guides the decisions

The purpose is to create useful wealth and share the means to create it. Public contributions should be practically usable under real permissions, including for commercial work. Private user projects can coexist with the commons.

Origins, uncertainty, responsibility, and rights should remain visible through the work. An estimate can be useful without becoming a measurement. A successful prediction can be informative without proving every assumption. A signature can identify an approval without proving the underlying claim.

One foundational hypothesis is that reusable scientific tools and records help a capable AI produce more dependable work with less repeated effort. We will compare the specialized workflow with a competent general agent using the same underlying model and comparable resources. Better quality, lower cost, and the value of specialized agents are things to test; none follows automatically from the architecture.

The nonprofit is a proposed legal and economic support for the shared project. The founder's possible company is an example of participation. Neither should organize the software around special privileges for one business.

How much of the vision is fixed

This plan treats the revised white paper as the current direction. It separates that direction from a recommended build sequence and open implementation choices. When new evidence changes a decision, record the change and retire conflicting guidance.

An old instruction should not become permanent merely because it has accumulated in project documents. Equally, an implementation shortcut should not silently change a public promise or a scientific distinction.

2 What we appear to have now

A useful starting point

A bounded read-only check confirmed that the repository's main version remains d19224f, the same version used in the revised white paper. The sources show useful implementation foundations. They do not establish that every connected behavior works in a running application. [1]

Area	What the inspected code contains	What still needs demonstration or development
Postdoc	A bounded tool loop, limits, fallback, and checked answers	The proposed richer coordination and evaluated task routing
Sources and results	Literature work and saved calculations with input and method records	One dependable investigation linking the work across the user's journey
Process engineering	Biological and process calculations, vessel sizing, recovery trains, and economics	Scientific validity within stated conditions and complete connections in use
Physical observations	Browser capture and comparison with predictions	A shared return path that makes supported measurements reusable
Equipment	Equipment names in procedures	Designs linked to particular machines, construction, condition, and maintenance
Sharing and release	Parts of the proposed foundation	Complete project permissions, deliberate publication, and adopted public terms

We should test and reuse the existing recovery-train Builder, proposal tools, saved results, and composite economics. They should not be scheduled as though nothing has been built.

The first technical investigation must run the relevant software, identify existing failures, show what survives reopening, and establish a recoverable starting version. That work has not been performed by this document's source review.

A conflict we need to resolve deliberately

The present Postdoc instructions and answer checks forbid model-written process numbers and estimates. The revised vision allows an AI estimate when its origin and limits remain visible. [2]

Changing a sentence in Postdoc's prompt would leave other parts expecting the old rule. The eventual change needs to align the way values are represented, saved, checked, used, displayed, and exported. Evaluations must check the new distinction between a supported estimate and a falsely presented measurement.

Existing checks that enforce the old behavior may need deliberate replacement. The team should explain which requirement changed and provide stronger relevant checks. It must not simply remove failing checks to make a new feature appear successful.

Two other connections need particular attention

The browser can record reconciled measurements, but the inspected service explicitly describes those browser records as unsynchronized. Saving something on one device is not the same as making it available to the authorized shared project. [3]

Existing model configuration includes a primary model, fallback, roles, and spending information. That is a useful foundation. It is not the evaluated task selection across models and providers described in the white paper. A small measured routing capability belongs early in development. [4]

What the new scientific case adds

BioSTEAM offers reusable process and economic modeling foundations. Its comparative example includes corn stover with dilute-acid pretreatment and switchgrass with AFEX. Those examples do not establish a calibrated model for every crossed feedstock/pretreatment pairing. The tutorial's illustrative uncertainty assumptions are not experimental uncertainty evidence. [5]

The historical NREL corn-stover design is a useful reference with mixed operating modes and a stated historical economic basis. It is not a ready-made batch-only plant or a current price quotation. BioSTEAM already includes batch-sizing tools; inspect and reuse appropriate components before proposing a new engine. [6]

None of those dependencies was installed or executed in preparing this update. The baseline should record versions, permitted reuse, operating assumptions, and the smallest reproducible reference. Reproducing software output, reconciling it with a published design, and checking prediction against independent observations are separate tasks.

3 One example we will follow

The recommended starting question

Use a calculation already supported by the software to examine a narrow part of a documented corn-stover ethanol scenario. A suitable question is: for a fixed, stated conversion scenario, how does changing one assumed product recovery affect the estimated amount remaining, and which missing information limits that answer?

This is a planning example, not a selected experimental protocol or a claim about real equipment. A product balance can be checked without constructing a new pretreatment or biological model. The baseline review should choose the smallest existing case that supports a clear answer. If the needed conversion model is absent, preserve a documented fixed conversion input and state that boundary. You should confirm the question's usefulness; the team chooses and explains the technical details.

The scientific boundary

The flagship requires batch biological conversion. The first case brief names the hydrolysis and fermentation stages included in that batch block; it does not silently require every upstream operation to be batch. Charging, processing, transferring, emptying, cleaning, vessel use, and downtime must be stated where they affect output. Recovery, finishing, and utilities may operate differently; record each operation's mode and the assumptions connecting them. Do not call a mixed-mode reference a fully batch plant, or require a new batch distillation model merely to use it as a reference.

A feedstock record needs sample or lot identity, composition basis, units, analytical method where known, and unresolved fractions. Dry and as-received values are different. Composition alone does not determine digestibility, inhibitor response, strain behavior, or successful conversion. Missing characteristics stay missing. A range of conditional scenarios is useful when a predictive model is not supported. [7]

Start with one documented feedstock, pretreatment, organism, and operating-condition pairing. Add another only with appropriate evidence. Vary operating conditions only when an applicable response relationship connects them to performance. An assumed yield is not automatically a controllable optimization variable.

The output should say best among the supported, feasible alternatives under the recorded assumptions. A conversion-only result or a provisional recovery factor is not a complete plant design. Plant-wide minimum selling price requires a consistent economic boundary, product specification, usable annual output, timing, recovery, utilities, waste handling, capital, and financial assumptions. Missing parts cannot silently become free.

The journey through the platform

You name the investigation and explain the decision you want to make. Postdoc finds relevant existing work and proposes a bounded analysis. You can inspect the sources and recognize which inputs are measured, quoted, calculated, estimated, or assumed.

The analysis runs through a supported calculation. You can follow the result to its exact inputs, method, and conditions. You close the application and later recover the work. Changing an input produces a linked revision while the old result remains understandable.

Postdoc may propose a variation alongside that work. Its record explains the idea, basis or absence of a traceable source, applicable conditions, assumptions, and proposed check. A supported calculation can screen it; unsupported parts remain research questions. Evaluation can reject, revise, or retain it without deleting its history.

When a real test becomes appropriate, the team records its procedure, actual machine, expected outcome, and planned observations before the work. Actual results return to the investigation. Postdoc explains what matched, what did not, and which questions the test cannot answer.

Equipment and economic records deepen the same example. A particular machine has its own construction and operating history. A permitted release gives another team enough information to inspect and repeat the supported work. A later reproduction or increase in capacity supplies its own evidence.

What counts before a new experiment exists

Example data can exercise the software when clearly identified. Historical observations can be imported with their real origins. Neither should be described as a new experiment performed through openFerment.

A useful software release can precede physical proof if it states that boundary honestly. Work that does not depend on a new experiment can continue while physical arrangements are prepared. The evidence that remains missing must stay visible.

4 The promises we must preserve

The user directs the work

The user chooses goals and limits. Routine searches, calculations, and drafts can proceed within delegated authority. Spending, record changes, release, and physical control have permissions appropriate to their consequences. The application must enforce those permissions rather than relying on a model to remember them.

Information keeps its meaning

Important inputs retain their origin, units, conditions, and limitations. Calculations preserve their input choices and versions. Connected tools agree about streams, composition, units, time, and scale. Charts and exports carry the context needed to interpret them.

Confidence should reflect relevant evidence and methods. One universal trust score is not a substitute for explaining why information applies to a particular decision.

Keep origin, applicability, and review status separate. A published measurement can be well sourced yet poorly applicable to a new feedstock. An expert-reviewed estimate is still an estimate. A calculation does not convert its assumed inputs into observations.

Distinguish real material or process variability from gaps in knowledge, measurement error, and limitations of the model itself. Use unknown, a conditional scenario, a supported interval, or an evidence-based probability distribution as appropriate. Preserve important dependencies: composition fractions must respect closure, and related yields cannot always vary independently. Do not invent a numerical confidence or a physical probability from an LLM's confidence, sampling temperature, or frequency of generating an idea.

Ideas have a lifecycle

Give a hypothesis an identity, generating context, model and instruction version where available, and an explicit next check. Keep the original proposal and the reasons for later changes. Record whether it is proposed, screened within model limits, awaiting evidence, contradicted, or supported under stated conditions. Approval to investigate is not scientific validation. Claims of novelty need appropriate review.

The scientific library is deliberate

The initial evidence base is a small, curated, versioned set of permitted papers, extracted data, equations, reference cases, and installed tools. Updates do not silently rewrite old results. Retrieved or imported text is evidence to inspect, not authority to change system permissions.

Postdoc uses an authorized model API. It has no independent web-search capability by default; simulation and optimization components do not perform research or fetch arbitrary content. User-requested acquisition or deliberately imported literature can update the library through a recorded process. Record source, access and reuse rights, extraction, review, and library version before relying on new material. Controlled acquisition is separate from model inference and does not imply locally hosted inference.

Useful work can be recovered

The investigation preserves the question, decisions, supporting work, and unresolved issues. Corrections remain attributable. Earlier results do not quietly change when someone edits an input. Recovery and export should be demonstrated, not inferred from the presence of a save button.

Real outcomes support limited conclusions

Predictions are recorded before outcomes. The project retains misses and inconclusive results as well as successes. A final result does not automatically validate all upstream parameters. A machine or process retains the conditions and scale at which its performance was demonstrated.

Public knowledge is deliberate and usable

Private projects remain private within the supported operating arrangement. Public contributions carry attribution, useful source files, and actual permissions. Training on project data needs its own authority. Openness of the software does not automatically make every user's work public.

We preserve the ability to change course

The team should justify major changes, preserve recoverable versions, and update the record of decisions. New tools, providers, agents, and domains must show useful behavior and evidence. Detailed architecture, legal terms, and future model-training methods remain choices to investigate.

5 The build sequence

Milestone 0 Know the starting point

Purpose Establish what we can safely build on.

You receive A running demonstration of the chosen path and a short account of what works and was checked, what is present but unverified, and what is missing. If the application cannot run, the team identifies the smallest relevant blocker. Important claims link to evidence. Select the first corn-stover question and curated reference set; inspect the applicable BioSTEAM dependency and record its versions and operating boundary without making an entire upstream installation a hidden prerequisite for the first small calculation.

You review Open the current example, follow its inputs and result, close and reopen it, and see the actual limits. Review the recommended first question and the first assignment.

Completion evidence A recoverable code and data version, relevant baseline checks with pre-existing failures distinguished, and a documented supported way to run the demonstration. Confirm actual API access and the library/network policy. Preserve one competent general-agent reference attempt and an independently checkable engineering example when access permits; record missing access honestly rather than substituting a scripted success.

Still outside this milestone Product implementation, a general rewrite, new infrastructure by default, or claims of physical validation.

Milestone 1 One investigation that stays understandable

Purpose Make the first useful connected software workflow dependable.

You can do State the bounded corn-stover question, preserve sources and labeled estimates, use an existing calculation, inspect the result's basis, return later, and export an understandable account. Postdoc helps with a bounded job and can preserve a clearly separate hypothesis. Unsupported feedstocks or operations receive an explanation of the missing evidence rather than an invented prediction.

You review Follow the example from question to result. Inspect a source and an estimate. Close and reopen it. Amend an input and compare the old and new result. Ask Postdoc to explain an unsupported conclusion and observe how it handles the gap. Cancel a task and reach a spending limit; see partial work and incurred costs preserved without unauthorized continuation.

Completion evidence Checks of saving, exact references, revisions, origins, units, errors, and export. A few representative task routes are evaluated for quality, total cost, time, and failure behavior. Evaluation examples are kept separate from examples used to tune routing. Record which model and tool versions were tested, and recheck important routes when they change. The user can see the chosen route, its costs, and its limits. Privacy and action boundaries match the actual deployment.

Still outside this milestone Hundreds of agents, a universal confidence score, general model training, physical truth, or every scientific module connected at once.

The creative lane needs useful proposal records and a bounded check, not a machine for producing endless variants. The first efficiency comparison needs matched resources and a readable resource record, not a large benchmark campaign. Both should deepen the connected investigation rather than become separate platforms.

Assignments should be small enough to examine. Section 6 prepares the first. Likely following assignments connect Postdoc to the records, add a small evaluated routing choice, and complete the whole milestone demonstration. Routing and evaluation are first-release requirements, not an indefinitely postponed optimization.

Milestone 2 Predictions and observations meet

Purpose Complete the return from work performed to evidence the project can use.

You can do Prepare a procedure and prediction, identify the equipment and conditions, capture observations and deviations, compare outcomes, and find supported measurements from the authorized project record afterward.

You review Record a prediction, capture or import clearly identified observations, amend a reading visibly, and inspect a comparison that includes a miss or inconclusive outcome. Show that matching final output does not magically turn an estimated parameter into a measurement.

Completion evidence The software return path works after reopening and in its supported shared setting. Evidence preserves the method, samples, machine, conditions, deviations, and prediction links. Interruption and synchronization checks fit the promised operating mode.

Physical proof is separate A subsequent real run requires responsible people, appropriate equipment, a reviewed procedure, and actual observations. Software completion and the physical demonstration receive separate status. A historical import proves neither a new experiment nor independent predictive performance.

Equipment identity is required here. Milestone 3 expands that record; it does not introduce the idea of an actual machine for the first time.

Milestone 3 Connect design to a real machine

Purpose Make equipment knowledge useful for building, operating, and improving a process.

You can do Follow a process requirement to editable design files, fabrication information, an actual built unit, commissioning, operation, faults, maintenance, and revisions. Use existing design tools rather than recreating them inside openFerment.

You review Two unit records share one design but retain separate construction differences and operating histories. Clearly labeled example records can first demonstrate that distinction; two physical builds are not required by this software demonstration. For the actual machine, a run names its real unit and configuration. A fault or measured limitation leads to a reasoned change without erasing the earlier record.

Completion evidence A usable package for one concrete machine, reviewed by people competent to fabricate and operate it. Relevant records include materials, substitutions, interfaces, tolerances, joining, tooling, assembly, inspection, utilities, controls, cleaning, calibration, maintenance, and operating limits. Actual physical claims require actual work.

Still outside this milestone A universal equipment catalogue, automatic certification, unrestricted machine control through conversation, or a claim of industrial readiness based on a drawing.

Milestone 4 Another person can inspect and reuse the work

Purpose Make collaboration and deliberate public release dependable.

You can do Give a collaborator appropriate access, preserve who changed or reviewed work, record disagreement and corrections, prepare a release, and inspect its contents and permissions. Someone unfamiliar with the project can understand its important assumptions and repeat the supported calculation.

You review One collaborator can perform a permitted action; another account cannot obtain private work through search or Postdoc. Export the package and have a new person use it. A restricted attachment is excluded without pretending the package includes rights it lacks.

Completion evidence Technical checks of access across the relevant routes, an inspected and usable export, a deliberate release record, and appropriate adopted terms for the material being shared. Publication requires actual release authority and rights review.

Still outside this milestone A claim that any open licence forces all experiment results upstream, that a published report clears patents, or that one signature proves technical truth.

If shared accounts or external access are introduced sooner, their access-control work moves sooner. This milestone adds fuller collaboration and release; security is a dependency of the setting in which the software is used.

Milestone 5 Reproduce and increase capacity with evidence

Purpose Find out whether the knowledge transfers, then investigate larger output as a separate question.

You can do Help another team repeat the work, preserve their adaptations and questions, and compare conditions, quality, reliability, resource use, labor, and costs. Record any proposed capacity increase separately from demonstrated performance.

You review What did the receiving team need that the first package omitted? Which substitutions worked? What failed? Which results agree under comparable conditions, and which remain uncertain?

Completion evidence Actual reproduction records from the receiving team. Increasing capacity needs its own relevant calculations, review, and physical evidence. Depending on the process, mixing, heat and mass transfer, recovery, cleaning, scheduling, waste, downtime, and economics may change.

Still outside this milestone Calling a larger design validated before it is tested, or treating one successful reproduction as proof that the process works everywhere.

Continuing development Expand useful capability

Additional biological tools, recovery operations, economic models, providers, training, team features, and engineering domains should follow the same pattern: a useful question, bounded work, relevant evidence, and an honest claim.

For the ethanol case, suitable later assignments may add a supported response relationship, limited operating-variable search, batch timing, sensitivity, and a second feedstock pairing. Switchgrass remains conditional. Arbitrary mixed waste, generalized inhibitor prediction, automated global flowsheet synthesis, all pretreatment methods, complete dynamic scheduling, detailed environmental claims, and autonomous experiments belong outside the initial commitment. A broad market-use question such as what to do with alfalfa must not silently become an ethanol-only optimization.

Teaching and competence support can develop alongside the workflow where users need them. Operators and fabricators must have ways to contribute corrections and receive credit. They should not wait for a distant final phase to be heard.

Learning begins with a visible track record, corrected inputs, useful retrieval, better route selection, and better experiments. Fine tuning or reinforcement learning is a later option if permitted data and independent evaluations show a benefit. No fixed number of experiments makes it necessary.

The milestone order reflects dependencies, not an artificial ban on parallel work. Equipment preparation, legal stewardship, and suitable learning materials may proceed alongside software where useful. No calendar or delivery budget is promised before the baseline and work estimates exist.

6 The first prepared assignment

Keep one investigation understandable when I return

This is the recommended first implementation assignment after Milestone 0. Parts already meeting the requirement should be kept. The technical approach will be designed later.

Why it matters Postdoc cannot be a dependable research coordinator when its answer outlives the records explaining it. One small saved investigation gives later agent work and measurement return something coherent to connect to.

The user's situation I have a narrow corn-stover ethanol question, a documented starting quantity, and an estimated conversion or recovery input. I want to use an appropriate input in an existing supported calculation, understand why it was used, and recover that explanation when I return. A simple product balance is enough; it does not establish the biological conversion or the physical recovery design.

What I should be able to do

1. Name the investigation and state its question, corn-stover reference conditions, process boundary, and intended decision. Identify batch conversion and state any separate recovery assumption.
2. Keep a sourced value and an estimated value from the curated reference set or user input. See units, composition or quantity basis, origin, applicability, review status, responsible person or tool where applicable, and important limitations. Unknown information remains unknown. A numerical range is included only when there is a defensible basis for it.
3. Use a suitable estimated input in one existing supported calculation. Open its result and follow it to the exact input and method version used. Its estimated origin and limitations remain visible in the result and export. Keep a sourced input in the same investigation so the distinction can be inspected.
4. Close and reopen the application and recover the work in its supported operating arrangement.
5. Amend an input and make a new result while still being able to inspect the earlier input and result.
6. Export an understandable account containing the question, selected inputs, origins and limitations, calculation reference, and relevant versions.

The demonstration you will review

Use one clearly identified example. Walk through the six actions in order, then let you find the source or estimate's basis without being coached through obscure menus.

Show the old and new results together. Explain whether a changed input affects other saved results and how the user can recognize that they may need review.

Then show missing required information, incompatible units, and a failed save. The application should explain what was not saved and what can be retried or recovered. A result should not appear saved if it exists only on the current screen.

Evidence the team supplies

The team checks recovery, exact references, units, revisions, failed operations, and the contents of the export. It checks that the task does not expose unrelated or private material under the actual workspace and account arrangement.

If the task changes a scientific calculation, it needs an appropriate reference check. If the calculation is unchanged, identify the existing verification and its limits. This assignment does not establish the science anew merely by preserving the calculation's inputs.

The completion note identifies what was already working, what changed, what was demonstrated, what remains unverified, and how to recover the completed version.

What this assignment does not require

It does not require a new database, a universal knowledge graph, every scientific module, team collaboration, public release, model training, or operating equipment. It does not require Postdoc to independently select a scientifically valid parameter.

It also does not require a new ethanol kinetic model, BioSTEAM integration, a batch scheduler, a pretreatment comparison, plant-wide economics, or the construction of a complete ethanol process. Upstream reference adoption and reproduction receive their own bounded assignment after the baseline. Existing calculations should supply this assignment's demonstrable result.

The estimate in this first assignment can be entered by the user. Align checks on the paths actually changed, without disguising an estimate as a measurement. Model-generated estimates and their wider Postdoc policy change belong in the following bounded assignment within Milestone 1. This keeps the first assignment small while preserving the first release's intended behavior.

An export here is a user-controlled copy, not automatic publication. The first example should use permitted data, with illustrative or historical material labeled accurately.

What needs to be settled before implementation

The team should recommend the supported calculation, the smallest coherent record change, and the recovery and export approach after the baseline check. You confirm the usefulness of the example and the behavior described above. You need not choose storage products or programming frameworks.

If satisfying this assignment would require a much larger prerequisite than expected, the team should explain the practical reason and propose a smaller demonstrable step. It should not quietly turn the assignment into an architectural rewrite.

7 Your repeating workflow

Work in understandable rounds

1. Review the next useful outcome. Read its purpose, what you will be able to do, and what counts as completion.
2. Clarify the behavior if needed. Describe the difference you want without prescribing how to code it.
3. Authorize the bounded assignment or a coherent group of small assignments. Let the team make routine implementation decisions and repair relevant failures within that scope.
4. Try the demonstration. Compare what happens with the stated outcome, including relevant errors and recovery.
5. Review the evidence and limits. Get independent technical or domain review where the claim warrants it.
6. Record the result, retain a recoverable version, and select the next assignment using what was learned.

The unit of review is a useful outcome. You should not be asked to approve every file edit, test run, or routine repair. Equally, a large change in purpose, visibility, public rights, or ongoing cost is not a routine implementation choice.

Questions you can ask without reading code

- What can I do now that I could not do before?
- Where did this value or conclusion come from?
- What survives if I close and reopen the work?
- What happens when an input is missing, a tool fails, or the budget is reached?
- What exactly does this demonstration prove, and what does it leave untested?
- Could someone else understand and repeat the supported work from the records?

A useful report from you is: the estimate is still there, but I cannot find its basis. That identifies a real failure in the intended behavior. You do not need to name the faulty function.

Keep a small set of living records

Maintain one home for the vision and requirements, one current-state account, one roadmap, important decisions, and the active assignment. They can start as sections of this plan and later move into suitable project documents. Avoid maintaining several conflicting copies.

For each assignment, record its outcome, status, evidence, remaining limits, and next step. Distinguish planned, in progress, demonstrated as software, and supported by physical evidence where relevant. A single completed label should not imply every kind of proof.

When a new idea appears, decide whether it clarifies the active assignment, fixes a defect, changes direction, or belongs in future work. Add future work to the roadmap rather than expanding every active task.

8 Decisions and recommended defaults

Start with one supported investigation

Default Choose one existing balance, conversion, or recovery calculation for a documented corn-stover scenario and a narrow useful question. Use permitted example data first. Conversion is batch; state recovery and utility modes separately. Switchgrass follows only if suitable evidence and software support it. Preserve the larger production and manufacturing ambition in the coverage map.

Who decides The team recommends a technically suitable example. You judge its relevance. A new physical procedure requires the people responsible for that work.

Preserve useful implementation

Default Extend existing records and tools where they can support the outcome. A general Study concept describes a connected investigation; it does not predetermine a new database, service, or set of screens.

When to reconsider The baseline identifies a concrete reliability, recovery, concurrency, or usability limit that cannot reasonably be addressed within the existing approach. The team explains the user effect and migration implications.

Make estimates useful and honest

Default Allow clearly identified estimates with a stated basis, conditions, and limitations. Preserve their origin in later results. Separate origin, applicability, uncertainty and review. Use scenarios and unknowns where formal distributions are not justified, retain relevant dependencies, and never treat LLM sampling probabilities as physical uncertainty. Keep generated ideas as hypotheses until evaluated.

When to reconsider A specific task needs a more formal decision rule or uncertainty method. It should be justified and evaluated for that task.

Begin routing with a few meaningful choices

Default Use ordinary software for suitable defined operations and evaluate a small number of useful model routes. Record the complete cost of checking and retries as well as the initial call. Retain a dependable comparison route. A fallback must respect the same permitted providers, data handling, and spending limits; it cannot silently expand them.

When to reconsider A new model, task, or privacy requirement needs evidence of a better route. Fine tuning and large agent teams require demonstrated benefit, not a preference for a more elaborate architecture.

Let the simple system compete

Default Test the same underlying model on the same bounded engineering questions with comparable evidence, installed scientific libraries, permissions and aggregate spending limits. The general agent must be competent and allowed to use scientific software. Count every specialist call, retry, check and simulation. Begin with a small comparison between real existing paths, using separate development and reserved examples.

What we learn Adding checked packaged tools tests their value. Comparing minimal Postdoc with that same path plus a specialist tests the specialist. If two proposed conditions are actually identical, merge them. Do not build four elaborate systems just to populate a table. Evaluate different model-routing policies separately so stronger models do not masquerade as an architectural gain.

When to expand A pilot identifies a worthwhile decision, observable variability and an affordable comparison. Choose case counts, repetitions and improvement targets from that evidence. No fixed percentage saving or quality improvement is promised. An efficient simple path is a valid result. [8]

Keep acquisition controlled

Default Postdoc uses an authorized API and the versioned project library. Independent web search is off by default. A user-requested acquisition or deliberate import can add sources with visible origin, rights, review and version history. Scientific tools operate on permitted installed resources.

When to reconsider A specific research question justifies a bounded acquisition capability. Its authority, evidence changes and spending must be explicit. This does not authorize every agent or simulator to browse or install software.

State the actual privacy and authority boundary

Default The first demonstration has one authorized project and a stated operating arrangement. Project ownership, visibility, permitted providers, budgets, and consequential actions are explicit. Only necessary and permitted information leaves the project.

When to reconsider Adding another account, remote access, shared hosting, publication, or a new provider changes the boundary. The relevant controls must be established before that change exposes data or grants authority.

Adopt public terms before public release

Default Prepare a clear account of what will be shared, under which rights, and with whose authority. Preserve the founding direction of public commercial reuse, suitable reciprocity, and no proprietary dual licensing of the common project. The legal mechanisms remain to be settled. Keep software, database contents, equipment designs, documentation, and patent questions distinct. Carry existing rights records forward without silently changing terms.

Who decides The accountable steward and rights holders adopt terms with appropriate advice. This plan does not select the legal home, final licence package, or patent policy. Administrative work can proceed alongside software development.

Explain costs before committing to them

Default Record which account pays for model tasks and which costs support shared infrastructure. Make task limits clear and show what happens when they are reached. Use existing authorized resources rather than assuming unlimited free compute.

When your input is needed A previously unauthorized recurring service, spending commitment, or material change to the economic arrangement. The team should present a recommendation and practical consequences, not a list of unexplained vendors.

Operational comparisons preserve observed model usage, calls, tool executions, retries, elapsed time, local simulation work, and actual or clearly estimated price. Hosted inference tokens are not joules. Record unavailable provider usage or energy as unavailable. Keep development, literature curation, maintenance, human review, precomputation and cache state separate from ordinary run cost. A subscription fee is not a precise per-task compute measure.

These defaults let technical work move forward. They are not substitutes for decisions about public commitments, sensitive information, physical work, or irreversible actions.

9 How we decide that work is complete

Match the evidence to the claim

Kind of evidence	What it can establish	What it does not establish by itself
Your demonstration	The behavior is usable and the explanation can be followed	Correctness of every calculation, permission boundary, or hardware design
Software checks	Specific saved behavior, errors, references, and boundaries match the checked cases	A real organism, machine, or factory performs as predicted
Scientific reference or expert review	A method or result is supported within the reviewed conditions	Performance outside those conditions or all future uses
Physical run	What was observed using the identified process and equipment	Independent accuracy of every parameter or general industrial readiness
Independent reproduction	Another team could repeat or adapt the stated work	That increasing capacity will work without further engineering

Evidence can be combined, but one kind should not silently stand in for another. A visual demonstration may reveal a problem even when automated checks pass. A physical result may reveal a limitation even when a simulation matches its reference case.

Require an understandable completion record

For each completed assignment, the team should state the intended outcome, what changed, what was actually checked, how you can try it, remaining limitations, and the next step. Claims that could not be verified remain marked as such.

Do not count a feature complete because a screen exists or code builds. Do not count an experiment complete because a file contains plausible numbers. Do not erase a failure by changing the expected answer without justification.

A mismatch may reveal an implementation defect, a bad test expectation, an unclear requirement, or a wrong scientific assumption. Diagnose which one it is and preserve the reason for the correction.

State what a comparison really shows

An independently checkable reference and hard feasibility requirements come before prose quality or a low modeled price. Apply a common process and economic boundary. A scenario may be correct under its assumptions without demonstrating physical performance. Hypothesis usefulness receives its own assessment; it cannot cancel a failed balance or fabricate missing costs.

Report successful, failed, incomplete and budget-exhausted attempts. Preserve the work used to select an answer. Use reserved cases and repeated trials as resources permit; record sampling and stopping rules before inspecting the result. Small pilots can guide development but do not establish universal superiority. Compare more than one meaningful allowance when affordable, because a baseline may improve with more work. [8, 9]

Show quality and measured cost separately. Similar quality at lower cost, better provenance at higher cost, task-dependent advantages, and no demonstrated advantage are all possible outcomes. Include cost per attempt and per verified success with honest denominators. Report cold versus repeated use, cache and precomputation treatment, and the human work required. Only a demonstrated improvement on both the agreed quality and resource measures supports a dominance claim for the tested task population. Do not claim total compute or energy savings from incomplete hosted-provider measurements.

Set an honest first release boundary

The first useful software release should demonstrate Milestone 1 on its supported example. It should include publicly available source code under its adopted terms, a documented route to independent use, installation instructions, recoverable work, usable export, contribution information, responsible maintenance, and stated limits. Hosted access can supplement that package. It does not replace it.

If it is shared with external users, the relevant account, privacy, and provider arrangements must already work. Public release needs adopted permissions and an accountable operating arrangement. The software can be useful without claiming a new physical experiment or a complete production line.

10 What we prepare later

The next technical planning stage will turn the first agreed assignment into a concrete implementation brief. It will choose the affected parts of the repository, data changes if needed, compatibility approach, appropriate checks, and a way to recover from a failed change.

Only then should detailed Codex execution prompts be written. They should refer to the current requirements, decisions, assignment, and actual implementation. They should not copy the entire history into every prompt or freeze hundreds of future tasks before their dependencies are understood.

Postdoc's own research instructions are a separate product responsibility. Changes to them should be versioned and evaluated alongside the tools and permissions they depend on. Writing the development prompt does not implement those capabilities.

For now, the reviewable result is the plan: one starting investigation, explicit requirements, an order of proof, recommended defaults, and a first assignment you can judge. The runtime baseline remains the first practical step before implementation.

The revised white paper's flagship and comparison sections provide the scientific and evaluation direction. They do not expand the first assignment into a complete plant, a research publication, or a large benchmark. The later team handoff should include repeatable reference cases, raw results or readable traces, known confounds and unresolved scientific questions as well as working software.

Coverage map

This map keeps the full vision visible. References identify white-paper sections and the earliest milestone where evidence should begin, not where the whole outcome is finished. M1 means milestone 1. Software and physical evidence remain distinct where both are needed.

Outcome	What we should be able to show	White paper sections and milestone
R01 User direction	Set or change a goal, inspect a plan, delegate within limits, and stop work	2 and 6; M1
R02 Connected investigation	Follow a question through its supporting work to a decision; a research-only result can finish honestly	3 and 4; M1
R03 Lasting work	Reopen work, preserve old versions, and recognize results affected by a changed input	4; M1
R04 Inspectable sources	Check an extraction against its actual source and retain access and reuse information	4 and 5; M1
R05 Distinct origins	Recognize measurements, quotations, calculations, estimates, and assumptions in results and exports	5; M1
R06 Applicability and uncertainty	Explain transfer between conditions and examine which uncertainty changes a decision	5; M1 onward
R07 Meaningful tool connections	Preserve units, streams, composition, time basis, and scale across linked calculations	4 and 7; M1 onward
R08 Bounded delegation	See useful tasks, progress, reviewed results, failures, and cancellation	6; M1
R09 Evaluated routing	Compare routes on separate evaluation examples; retain tested versions, quality, total cost, time, provider restrictions, and fallback	6; M1
R10 Enforced authority	Check that conversation or retrieved text cannot grant unauthorized reading, spending, writing, or release	6 and 12; M1 and each expansion of access
R11 Useful memory	Recover prior reasons and unresolved questions without leaking another project's information	2 and 6; M1 onward
R12 Biological tools	Inspect assumptions behind biological constraints and candidates; distinguish theory from measured performance	7; M0–1, then extensions
R13 Process and reactor models	Inspect fitted data, assumptions, conditions, and relevant reference or predictive checks	7; M0–1, then M2
R14 Recovery and production	Follow a harvest into recovery and finishing, with losses, utilities, waste, and missing inputs visible	7; preserve at 0, connect when required
R15 Economics	Distinguish estimates, quotations, and actual costs; retain place, date, currency, and important operating assumptions	7 and 8; M1 through 5 as relevant

Outcome	What we should be able to show	White paper sections and milestone
R16 Buildable equipment knowledge	Provide editable files, fabrication and assembly information, components, interfaces, and support methods	8; M3, preparation can begin earlier
R17 Actual machine history	Distinguish two built units and their substitutions, commissioning, configuration, calibration, and maintenance	8 and 9; identity at 2, lifecycle at 3
R18 Scale and reproduction	Keep proposed capacity, demonstrated conditions, and a second site's results separate	8; M3 and 5
R19 Prepared physical work	Save procedure versions, responsible people, equipment, predictions, and amendments	9; M2
R20 Actual capture	Preserve readings, samples, imports, deviations, corrections, and interruptions without inventing observations	9; M2
R21 Reusable measured evidence	Recover supported observations through the authorized project after the run and supported synchronization	9 and 13; M2
R22 Honest calibration	Compare supported, missed, and inconclusive outcomes without validating untested parameters	5 and 9; M2
R23 Inspectable improvement	Explain what changed after evidence, evaluate it on fresh cases, and recover an earlier version	9; M2 onward
R24 Deliberate sharing	Control access, publication, model-training permission, disagreement, and linked corrections	10 and 11; foundations at 1, fuller experience at 4
R25 Usable public rights	Identify what can be reused, modified, manufactured, and sold and which rights remain unresolved	10; prepare early, adopt before release at 4 or sooner
R26 People and learning	Connect roles, observed competence, teaching, operator contributions, and attribution to real work	11; develop alongside relevant milestones
R27 Recovery and portability	Demonstrate export, usable receiving formats, backup recovery, and understandable service dependencies	12; first path at 1, independent use at 4
R28 Sustainable public project	Identify maintainers, operating costs, public limitations, contribution routes, stewardship, and open decisions	1 and 13 through 15; prepare throughout, establish for release
R29 Bounded ethanol flagship	Trace a supported corn-stover batch case with declared recovery, evidence, uncertainty and hypotheses; add switchgrass only with support	16; M0–1, then extensions
R30 Comparative value	Repeat a competent matched baseline, record verified quality and observed cost separately, and test added orchestration without assuming advantage	17; small comparison at 1, evaluated throughout

Sources and status

The revised openFerment white paper supplies the product direction, including its new flagship and comparative-evaluation sections 16 and 17. This plan version 0.2 integrates the supplied discussion with the clarified batch-conversion and API/network boundaries. It translates the scope into outcomes, proposed milestones, decisions, and a first assignment. The sequence and defaults are recommendations for review rather than an adopted technical architecture.

[1] Repository snapshot confirmed October 21 2026. Pinned main version. Representative foundations include computed result records, recovery train code, and composite economics.

[2] Current estimate restrictions appear in the Postdoc system prompt and answer contract and the bounded agent implementation. The revised estimate policy requires a deliberate change across the affected behavior.

[3] Browser records and the shared-evidence boundary appear in browser persistence, reconciliation, and the service's explicit partial coverage. Protocol equipment fields are a starting point, not a full machine history.

[4] Configured model roles and fallback show existing model-selection foundations. Live providers, availability, prices, and comparative task quality were not verified in the planning check.

[5] BioSTEAM comparative techno-economic analysis supplies reusable examples and explicitly qualifies its illustrative uncertainty assumptions. Reuse is a recommendation; neither those examples nor the first feedstock pairing has been executed here.

[6] The 2011 NREL corn-stover design report supplies a historical mixed-mode reference with stated scale and financial assumptions. BioSTEAM batch sizing documents existing batch-design support. Their presence does not establish the proposed process's validity.

[7] Biomass compositional analysis procedures support explicit analytical methods and basis. The separation between composition and conversion performance remains an important scientific boundary.

[8] AI Agents That Matter motivates cost-aware comparison, simple baselines, holdout data and reproducibility. It provides evaluation guidance, not evidence of openFerment superiority.

[9] AISI's discussion of test-time compute explains why measured capability depends on the allowed workload. Applying that principle here is a planning recommendation, not a measured bioprocess result.

The repository check was read-only and bounded; its pinned findings remain the October 21 source baseline. This October 22 update adds a source review and planning judgments, not a new repository execution audit. No application or tests were run, no dependency installed, no live-model task or physical experiment performed, and no benchmark, deployment or access-control boundary demonstrated. Code presence is separated from working behavior throughout the plan. Milestone 0 supplies the missing runtime baseline before implementation commitments are made.